From Bits to Pixels

The Mathematical and Physical Foundations of elementary 3D Graphics

Jack Wetherell

Contents

1 Foundations: Numbers, Memory, and Representation **3**
1.1 Bits, Bytes, and Binary Storage . . . 3
1.2 Floating Point Representation (IEEE 754) . . . 4
1.3 From Raw Memory to Numerical Values . . . 4

2 Geometry Representation in Memory **6**
2.1 Vertices (Position Vectors) . . . 6
2.2 Triangles (Faces) . . . 6
2.3 Indexed vs Non-Indexed Geometry . . . 7
2.4 Meshes (Objects) . . . 8
2.5 Scenes as Collections of Objects . . . 8

3 Basis Vectors and Coordinate Consistency **9**
3.1 Implicit Basis in Matrix Representations . . . 9
3.2 Consistency of Basis in Matrix Multiplication . . . 10
3.3 Local vs World Basis Interpretation . . . 10
3.4 Transform Chains as Basis Transitions . . . 11
3.5 Conceptual Summary . . . 12

4 Coordinate Systems and Spaces **12**
4.1 Local (Object) Space . . . 12
4.2 World Space . . . 13
4.3 Camera (View) Space . . . 13
4.4 Clip Space . . . 14
4.5 Normalized Device Coordinates (NDC) . . . 14
4.6 Screen Space . . . 15
4.7 Summary of Transform Pipeline . . . 15
4.8 Conceptual Interpretation . . . 15

5 Rasterization **16**
5.1 Triangle Setup . . . 16
5.2 Barycentric Coordinates . . . 16
5.3 Interpolation Across Surfaces . . . 17
5.4 Depth Testing (Z-Buffer) . . . 18
5.5 Summary . . . 19

6 Lighting and Surface Shading **19**
6.1 Surface Normals . . . 19
6.2 Light Direction . . . 20
6.3 Diffuse Lighting (Lambertian Reflection) . . . 20
6.4 Ambient Lighting . . . 20
6.5 Combined Lighting Model . . . 21
6.6 Transforming Normals . . . 21
6.7 Basic Shadows (Visibility) . . . 21
6.8 Shadow Mapping (Rasterization Approximation) . . . 22
6.9 Interpolation of Normals . . . 22
6.10 Summary . . . 22

7 Memory Layout of Graphics Data **23**
7.1 Vertex Buffers . . . 23
7.2 Index Buffers . . . 24
7.3 Transformation Storage . . . 24
7.4 Estimating Memory Usage . . . 25
7.5 Example: Pyramid . . . 25
7.6 Summary . . . 26

8 Specialized Hardware in GPUs **26**
8.1 Graphics Pipelines and Fixed Function Units . . . 26
8.2 Ray Tracing Cores (RT Cores) . . . 27
8.3 Tensor Cores and Machine Learning . . . 28
8.4 Evolution of GPU Specialization . . . 29
8.5 Conceptual Interpretation . . . 29

9 Shaders in Practice **30**
9.1 The ShaderLab File Structure . . . 30
9.2 Properties: Data Passed Into the Shader . . . 31
9.3 The SubShader and Pass . . . 32
9.4 The Vertex Shader (HLSL) . . . 32
9.5 Rasterization (Fixed Hardware Stage) . . . 34
9.6 The Fragment Shader (HLSL) . . . 34
9.7 Swizzling . . . 35
9.8 Data Flow Summary . . . 36

10 Unity: Unlit Shaders **37**
10.1 Basic Colour . . . 37
10.2 Vertex vs Fragment: Choosing Where to Compute . . . 41

1 Foundations: Numbers, Memory, and Representation

Before any triangle is drawn or any shader executes, the hardware must represent numbers. Everything in a computer, every vertex coordinate, every matrix entry, every color value ultimately lives as a pattern of electrical high/low signals stored in transistors. Understanding how numbers are encoded in this binary substrate is the starting point for understanding the entire graphics pipeline.

1.1 Bits, Bytes, and Binary Storage

At the lowest level, all data in a computer is represented using binary digits. A transistor can be either conducting or non-conducting, which maps cleanly to the two states of a binary digit.

- A **bit** is a single binary value:
$$b \in \{0, 1\} \tag{1}$$
- A **byte** consists of 8 bits:
$$1 \text{ byte} = 8 \text{ bits} \tag{2}$$

A sequence of bits can represent integers using positional notation in base 2. Just as the decimal digit in the "tens" column contributes a factor of 10^1, the bit in position i contributes a factor of 2^i:

$$N = \sum_{i=0}^{n-1} b_i \cdot 2^i \tag{3}$$

where:

- b_i is the i-th bit (0 or 1)
- $i = 0$ corresponds to the least significant bit (LSB), i.e. the rightmost bit

Example:
$$1011_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 8 + 0 + 2 + 1 = 11_{10} \tag{4}$$

With n bits we can represent 2^n distinct values, ranging from 0 to $2^n - 1$. A single byte therefore covers 0 to 255, which is why unsigned 8-bit colour channels run from 0 to 255.

Memory is a linear array of bytes, each uniquely identified by a non-negative integer address:

$$\text{Memory} = [\text{byte}_0, \text{byte}_1, \ldots, \text{byte}_N] \tag{5}$$

$$\text{Address} \in \mathbb{N} \tag{6}$$

The CPU and GPU access memory by supplying an address; the hardware returns the bytes stored at that location. Multi-byte values (such as 32-bit floats) occupy several consecutive addresses.

1.2 Floating Point Representation (IEEE 754)

Integers can be stored exactly in binary, but 3D graphics relies heavily on *real* numbers, positions, angles, colours. Real numbers cannot in general be represented exactly with a finite number of bits, so we use an approximation: **floating point**.

The IEEE 754 standard defines the format used by virtually every modern processor and GPU. The key idea is to store numbers in a form analogous to scientific notation: a sign, a significand (the meaningful digits), and an exponent that scales the significand by a power of two.

A 32-bit floating point number (single precision) is composed of:

- 1 bit for the sign (s): determines whether the value is positive or negative
- 8 bits for the exponent (e): encodes a power-of-two scale factor
- 23 bits for the mantissa (fraction) (m): encodes the significant digits

The value represented is:

$$x = (-1)^s \cdot (1 + m) \cdot 2^{e-127} \tag{7}$$

where the fractional part of the mantissa is:

$$m = \sum_{i=1}^{23} m_i \cdot 2^{-i} \tag{8}$$

Interpretation:

- $(-1)^s$ determines the sign: $s = 0$ gives $+1$, $s = 1$ gives -1
- $(1 + m)$ is the **normalized significand**. The leading 1 is not stored explicitly, it is always implied, giving 24 bits of effective precision for "free"
- $e - 127$ is the **unbiased exponent**. The bias of 127 means the stored value e represents the true exponent $e - 127 \in [-126, 127]$, allowing negative exponents to be stored as positive integers without a separate sign bit

Special Cases: Not all 256 values of e correspond to normal numbers:

- $e = 0 \rightarrow$ **denormalized numbers**: the leading 1 is *not* implied; used for very small magnitudes close to zero
- $e = 255 \rightarrow$ reserved: represents $\pm\infty$ (when $m = 0$) or **NaN** (Not a Number, when $m \neq 0$)

Single precision can represent magnitudes from roughly 10^{-38} to 10^{38}, with about 7 significant decimal digits of precision. This is generally adequate for geometry, but precision loss ("z-fighting") can occur when two surfaces are very close together in depth.

1.3 From Raw Memory to Numerical Values

All numerical values in a program are ultimately stored as sequences of bytes in memory.

A 32-bit float occupies 4 consecutive bytes:

$$\text{float} \rightarrow [b_0, b_1, b_2, b_3] \tag{9}$$

These bytes are interpreted according to the IEEE 754 structure:

$$\underbrace{s}_{1\text{ bit}} \quad \underbrace{e}_{8\text{ bits}} \quad \underbrace{m}_{23\text{ bits}} \tag{10}$$

Example: Representing $x = 7.98$ The decimal number 7.98 cannot be represented exactly in binary because its binary expansion is non-terminating (analogous to $\frac{1}{3}$ in decimal). The IEEE 754 representation therefore rounds to the nearest representable value:

$$x \approx 7.980000019073486 \tag{11}$$

Binary layout:

$$0\ 10000001\ 11111111001100110011010 \tag{12}$$

Breaking this down:

- Sign bit:
$$s = 0 \quad \text{(positive)} \tag{13}$$
- Exponent:
$$e = 10000001_2 = 129_{10} \tag{14}$$
$$e_{\text{unbiased}} = 129 - 127 = 2 \tag{15}$$
This means the value is scaled by $2^2 = 4$.
- Mantissa:
$$m = 1.11111111001100110011010_2 \tag{16}$$
With the implicit leading 1, the significand is approximately 1.995.

Reconstructing the value:

$$x = (-1)^0 \cdot (1 + m_{\text{fraction}}) \cdot 2^2 \approx 1.995 \times 4 \approx 7.98 \tag{17}$$

The small error ($\approx 2 \times 10^{-8}$) is intrinsic to the finite-precision representation.

Key Insight: There is no intrinsic meaning in memory, only interpretation. The same 32 bits could be read as a float, as an integer, as four colour bytes, or as part of a vertex. The *program* (or *shader*) decides how to interpret the raw bit pattern:

$$\text{Bits} \xrightarrow{\text{interpretation}} \text{Number} \tag{18}$$

The same sequence of bits may represent:

- an integer
- a floating point number
- part of a vertex
- or any structured data

Conclusion: All higher-level constructs (vectors, matrices, geometry) are built from this foundation:

$$\text{Bits} \rightarrow \text{Bytes} \rightarrow \text{Floats} \rightarrow \text{Mathematical Objects} \tag{19}$$

This chain is not merely theoretical, every vertex shader operand, every matrix entry in a uniform buffer, and every colour written to a framebuffer passes through exactly this representation.

2 Geometry Representation in Memory

Now that we understand how individual numbers are stored, we can build up the geometric structures needed to describe 3D scenes. The central challenge is to represent continuous geometry, surfaces, volumes, objects, using discrete data structures that can be stored in memory and processed efficiently by the GPU.

2.1 Vertices (Position Vectors)

A **vertex** represents a point in 3D space. It is the fundamental geometric primitive: all surfaces are built from vertices.

In its simplest form, a vertex consists of three floating-point values:

$$\mathbf{v} = \begin{bmatrix} x \\ y \\ z \end{bmatrix} \in \mathbb{R}^3 \tag{20}$$

For transformation purposes, vertices are often expressed in **homogeneous coordinates** by appending a fourth component $w = 1$:

$$\mathbf{v}_h = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \in \mathbb{R}^4 \tag{21}$$

The reason for this extra component is that translation , shifting a point in space , cannot be expressed as a 3×3 matrix multiplication. By working in 4D homogeneous coordinates, translation can be encoded as a 4×4 matrix, and all affine transformations (translation, rotation, scale, shear) can be composed by simple matrix multiplication. The convention $w = 1$ distinguishes *points* from *directions* (which use $w = 0$ and are unaffected by translation).

Memory Representation: Each coordinate is a 32-bit float (4 bytes):

$$\text{vertex} = [x, y, z] \tag{22}$$

$$\text{size of vertex} = 3 \times 4 \text{ bytes} = 12 \text{ bytes} \tag{23}$$

Thus, a vertex buffer is a contiguous array of these structures:

$$V = [\mathbf{v}_0, \mathbf{v}_1, \ldots, \mathbf{v}_n] \tag{24}$$

2.2 Triangles (Faces)

The fundamental rendering primitive is the **triangle**. While real-world surfaces are smooth and curved, the GPU approximates them with flat triangular facets. Triangles are chosen rather than other polygons for several important reasons:

- A triangle is always *planar* , three non-collinear points uniquely define a flat surface, so no special handling is needed
- A triangle is always *convex*, this simplifies rasterization (covered in Section 5)
- Any polygon can be decomposed into triangles (*triangulated*)
- Hardware can be optimized for a single primitive type

A triangle is defined by three vertices:

$$\triangle = (\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2) \tag{25}$$

Geometrically, any point inside the triangle can be expressed using **barycentric coordinates** (α, β, γ) as a weighted sum of the vertices. This is the natural coordinate system for a triangle:

$$\mathbf{p} = \alpha\mathbf{v}_0 + \beta\mathbf{v}_1 + \gamma\mathbf{v}_2 \tag{26}$$

subject to:

$$\alpha + \beta + \gamma = 1, \quad \alpha, \beta, \gamma \geq 0 \tag{27}$$

Each coefficient measures how "close" the point is to the corresponding vertex: $\alpha = 1$ gives exactly $\mathbf{v}_0$, and $\alpha = 0$ means the point is on the opposite edge. Barycentric coordinates are essential for interpolating per-vertex attributes (colours, normals, texture coordinates) across the triangle interior.

Memory Representation (Non-Indexed): Each triangle stores three full vertices:

$$\text{triangle size} = 3 \times 12 \text{ bytes} = 36 \text{ bytes} \tag{28}$$

2.3 Indexed vs Non-Indexed Geometry

In practice, adjacent triangles *share vertices*, every interior vertex of a mesh is shared by roughly six triangles. Storing each vertex explicitly per-triangle wastes memory and prevents the GPU from reusing cached vertex computations.

Non-Indexed Representation: Each triangle explicitly stores its three vertices:

$$T = [\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2] \tag{29}$$

This leads to duplication: a vertex shared by k triangles is stored k times. For a closed mesh this is typically a 6× redundancy.

Indexed Representation: Instead, vertices are stored *once* in a vertex buffer:

$$V = [\mathbf{v}_0, \mathbf{v}_1, \ldots, \mathbf{v}_n] \tag{30}$$

Triangles are then defined by integer *indices* into this buffer, rather than by copying vertex data:

$$F = [(i_0, i_1, i_2), (i_3, i_4, i_5), (i_6, i_7, i_8)] \tag{31}$$

The **winding order** of the three indices (clockwise or counter-clockwise when viewed from outside) specifies which face of the triangle is the front face. This is used for back-face culling: triangles facing away from the camera can be discarded before shading.

Each index refers to a vertex in the shared buffer:

$$\triangle_k = (\mathbf{v}_{i_a}, \mathbf{v}_{i_b}, \mathbf{v}_{i_c}) \tag{32}$$

Memory Efficiency: Let:

$$n_v = \text{number of vertices}, \quad n_t = \text{number of triangles} \tag{33}$$

Then:

$$\text{non-indexed size} = 36 \cdot n_t \tag{34}$$

$$\text{indexed size} = 12 \cdot n_v + 12 \cdot n_t \tag{35}$$

(assuming 32-bit indices, 4 bytes each, 3 per triangle)

For a typical closed mesh, $n_v \approx \frac{n_t}{2}$, so the indexed format uses roughly $12 \cdot \frac{n_t}{2} + 12 \cdot n_t = 18n_t$ bytes versus $36n_t$, a $2\times$ saving in vertex data, plus the GPU's post-transform vertex cache can avoid re-transforming shared vertices.

2.4 Meshes (Objects)

A **mesh** (object) groups a vertex buffer and an index buffer into a single logical unit:

$$\mathcal{M} = (V, F) \tag{36}$$

The mesh is defined in **local (object) space**, a coordinate system attached to the object itself. All vertices are expressed relative to the object's own origin, which is usually placed at a convenient point (the centre of mass, the base, etc.):

$$\mathbf{v}_{local} \in \mathbb{R}^3, \quad \mathbf{o}_{local} = (0, 0, 0) \tag{37}$$

Defining meshes in local space is practical: an artist models the object in isolation without worrying about where it will appear in the final scene. The object's position, orientation, and scale in the world are specified separately by a transformation matrix (Section 4).

2.5 Scenes as Collections of Objects

A **scene** consists of multiple objects:

$$\mathcal{S} = \{\mathcal{M}_1, \mathcal{M}_2, \ldots, \mathcal{M}_k\} \tag{38}$$

Each object has an associated 4×4 **model matrix** encoding its position, orientation, and scale in the shared world coordinate system:

$$M_i \in \mathbb{R}^{4\times 4} \tag{39}$$

which maps local coordinates to world coordinates:

$$\mathbf{v}_{world} = M_i \mathbf{v}_{local} \tag{40}$$

This separation is powerful: the same mesh can appear many times in a scene (trees in a forest, bullets in a game) simply by associating different model matrices with the same (V, F) data, a technique called **instancing**.

Global Interpretation: All objects share a common world coordinate system:

$$\mathbf{v}_{world} \in \mathbb{R}^3 \tag{41}$$

Thus, the scene becomes a unified collection of geometry in world space.

Final Structure in Memory:

$$\text{Scene} = \{(V_1, F_1, M_1), (V_2, F_2, M_2), \dots\} \tag{42}$$

Each component ultimately reduces to:

$$\text{Bytes} \to \text{Floats} \to \text{Vertices} \to \text{Triangles} \to \text{Meshes} \to \text{Scene} \tag{43}$$

3 Basis Vectors and Coordinate Consistency

All vector and matrix operations in graphics implicitly depend on a chosen basis. Understanding how basis vectors are represented and transformed is essential for interpreting coordinate systems and transformation pipelines. This section makes the implicit explicit: every set of coordinates is only meaningful relative to a reference frame.

3.1 Implicit Basis in Matrix Representations

A **basis** is a set of linearly independent vectors that span a vector space. In 3D, we need three basis vectors. Coordinates are then the scalar multiples needed to reconstruct a vector from those basis vectors:

$$\mathcal{B} = \{\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3\} \tag{44}$$

Then a vector $\mathbf{v}$ is written as:

$$\mathbf{v} = x\mathbf{e}_1 + y\mathbf{e}_2 + z\mathbf{e}_3 \tag{45}$$

In coordinate form, the components (x, y, z) are specific to the basis $\mathcal{B}$:

$$[\mathbf{v}]_{\mathcal{B}} = \begin{bmatrix} x \\ y \\ z \end{bmatrix} \tag{46}$$

Key Insight: The components (x, y, z) have meaning only with respect to a basis. The same vector $\mathbf{v}$ will have different numerical coordinates if measured in a different basis. This is why changing coordinate systems changes the numbers even though the underlying geometry is unchanged:

$$\text{Coordinates} \neq \text{Vector} \quad \text{(without basis)} \tag{47}$$

Matrices Encode Bases: A matrix transforms vectors by describing what happens to the basis vectors themselves. The columns of a transformation matrix are exactly the images of the standard basis vectors under the transformation. If the standard basis vectors $\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3$ are mapped to $\mathbf{c}_1, \mathbf{c}_2, \mathbf{c}_3$ respectively, then:

$$M = \begin{bmatrix} | & | & | \\ \mathbf{c}_1 & \mathbf{c}_2 & \mathbf{c}_3 \\ | & | & | \end{bmatrix} \tag{48}$$

where each column $\mathbf{c}_i$ represents a transformed basis vector. This follows directly from the definition of matrix–vector multiplication:

$$M\mathbf{e}_i = \mathbf{c}_i \tag{49}$$

To find where any vector goes under M, we decompose it in the basis and linearly combine the column images. Reading a transformation matrix is therefore equivalent to reading off its three column vectors and asking: "where do the three coordinate axes go?"

3.2 Consistency of Basis in Matrix Multiplication

Matrix multiplication is only valid when all operands are expressed in a consistent basis. Mixing coordinate systems without an explicit change-of-basis step is one of the most common sources of errors in graphics programming.

Given:

$$\mathbf{v}' = M\mathbf{v} \tag{50}$$

this implies that both M and $\mathbf{v}$ are expressed in the *same* basis $\mathcal{B}$:

$$[\mathbf{v}]_{\mathcal{B}} \to [\mathbf{v}']_{\mathcal{B}} \tag{51}$$

Both $\mathbf{v}$ and M must be defined relative to the same basis $\mathcal{B}$.

Composition of Transformations: For two transformations applied in sequence:

$$\mathbf{v}'' = M_2 M_1 \mathbf{v} \tag{52}$$

the intermediate result:

$$\mathbf{v}' = M_1 \mathbf{v} \tag{53}$$

must be expressed in the same coordinate system that M_2 expects as input. Concretely, if M_1 transforms from local space to world space, then M_2 must be a matrix that operates on world-space vectors.

Key Constraint:

$$M_2 \cdot M_1 \quad \text{is valid only if their bases are aligned} \tag{54}$$

In graphics, this constraint is satisfied by carefully tracking which space each matrix operates in (local, world, camera, etc.) and ensuring that each matrix in a chain maps from the output space of the previous matrix.

3.3 Local vs World Basis Interpretation

In graphics, different spaces correspond to different bases.

Local Basis: Each mesh is defined in its own local coordinate system:

$$\mathcal{B}_{local} = \{\mathbf{e}_x^{local}, \mathbf{e}_y^{local}, \mathbf{e}_z^{local}\} \tag{55}$$

Vertex positions are expressed as:

$$[\mathbf{v}]_{local} \tag{56}$$

World Basis: All objects share a single global coordinate system:

$$\mathcal{B}_{world} = \{\mathbf{e}_x^{world}, \mathbf{e}_y^{world}, \mathbf{e}_z^{world}\} \tag{57}$$

The model matrix maps between these two bases. Its columns tell us where the local axes point when the object is placed in the world:

$$\mathbf{v}_{world} = M_{model}\mathbf{v}_{local} \tag{58}$$

Interpretation of M_{model}:

$$M_{model} = \begin{bmatrix} | & | & | \\ \mathbf{e}_x^{local\to world} & \mathbf{e}_y^{local\to world} & \mathbf{e}_z^{local\to world} \\ | & | & | \end{bmatrix} \tag{59}$$

Each column describes how a local basis vector is expressed in world coordinates. If the model matrix is the identity, the local and world axes coincide. A rotation matrix rotates the local axes; a scale matrix stretches them; a translation shifts the origin.

3.4 Transform Chains as Basis Transitions

The full graphics pipeline can be interpreted as a sequence of basis changes. Each transformation matrix re-expresses vertex coordinates in a new frame of reference:

$$\mathbf{v}_{clip} = M_{proj} \cdot M_{view} \cdot M_{model} \cdot \mathbf{v}_{local} \tag{60}$$

Each matrix performs a change of basis:

- M_{model}: local $\to$ world (place object in scene)
- M_{view}: world $\to$ camera (move world so camera is at origin)
- M_{proj}: camera $\to$ clip (apply perspective distortion)

Thus:

$$[\mathbf{v}]_{local} \to [\mathbf{v}]_{world} \to [\mathbf{v}]_{camera} \to [\mathbf{v}]_{clip} \tag{61}$$

Key Insight: Because matrix multiplication is associative, the three matrices can be pre-multiplied into a single **combined transform matrix**:

$$M_{total} = M_{proj} \cdot M_{view} \cdot M_{model} \tag{62}$$

so that:

$$\mathbf{v}_{clip} = M_{total}\mathbf{v}_{local} \tag{63}$$

This pre-multiplication is done on the CPU once per object per frame, and the single matrix M_{total} is then passed to the vertex shader, which applies it to every vertex. This avoids repeating the multiplication of all three matrices for each of the potentially millions of vertices in the scene.

3.5 Conceptual Summary

- Every vector is defined relative to a basis; coordinates are meaningless without one
- Every matrix encodes a transformation of basis vectors, readable from its columns
- Matrix multiplication composes consistent basis transformations, the output space of one matrix must match the input space of the next
- The graphics pipeline is a sequence of basis changes, each serving a distinct geometric purpose

$$\text{Coordinates} + \text{Basis} = \text{Geometric Meaning} \tag{64}$$

$$\text{Transformations} = \text{Changes of Basis} \tag{65}$$

4 Coordinate Systems and Spaces

In computer graphics, geometry is transformed through a sequence of coordinate systems. Each space provides a different frame of reference for interpreting vertex positions, and each stage of the transformation pipeline serves a specific geometric purpose. The full sequence is:

$$\mathbf{v}_{local} \to \mathbf{v}_{world} \to \mathbf{v}_{camera} \to \mathbf{v}_{clip} \to \mathbf{v}_{ndc} \to \mathbf{v}_{screen} \tag{66}$$

4.1 Local (Object) Space

Local space (or object space) is the coordinate system in which a mesh is originally defined. It is the most convenient space for the artist or programmer who is building the mesh, since the object can be modelled symmetrically around its own origin without worrying about its eventual position in the scene.

$$\mathbf{v}_{local} = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \tag{67}$$

Key properties:

- Origin is the object's centre (or a chosen reference point):

$$\mathbf{o}_{local} = (0, 0, 0) \tag{68}$$

- Basis vectors define the object's natural orientation (e.g. $+y$ is "up" for the object):

$$\{\mathbf{e}_x^{local}, \mathbf{e}_y^{local}, \mathbf{e}_z^{local}\} \tag{69}$$

Vertices are stored in this space in memory. When the same mesh is reused at multiple positions (instancing), it is stored once in local space, and different model matrices place copies of it throughout the scene.

4.2 World Space

World space is a global coordinate system shared by all objects in the scene. It is the "ground truth" reference frame: positions of lights, cameras, and collision volumes are all expressed here.

Each object is placed into world space using the **model transformation**:

$$\mathbf{v}_{world} = M_{model}\mathbf{v}_{local} \tag{70}$$

The model matrix decomposes into scale (S), rotation (R), and translation (T) applied in that order:

$$M_{model} = T \cdot R \cdot S \tag{71}$$

The order matters because these operations do not commute. Scaling first, then rotating, then translating is the conventional order: it scales around the object's own origin, rotates the scaled object, then moves it to its world position. Explicitly, the combined matrix is:

$$M_{model} = \begin{bmatrix} r_{11}S_x & r_{12}S_y & r_{13}S_z & t_x \\ r_{21}S_x & r_{22}S_y & r_{23}S_z & t_y \\ r_{31}S_x & r_{32}S_y & r_{33}S_z & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{72}$$

where r_{ij} are the entries of the rotation matrix R and S_x, S_y, S_z are the scale factors along each axis.

Key properties:

- Common reference frame for all objects, lighting calculations and inter-object interactions happen here
- Fixed global basis, conventionally right-handed with $+y$ pointing up:

$$\{\mathbf{e}_x^{world}, \mathbf{e}_y^{world}, \mathbf{e}_z^{world}\} \tag{73}$$

4.3 Camera (View) Space

Camera space (or view space) represents coordinates relative to the camera. In this frame, the camera is always at the origin, looking along the negative z-axis, with $+y$ pointing upward. This canonical placement simplifies the perspective projection step that follows.

The key insight is that we do not move the camera to look at the scene, we keep the camera fixed (at the origin) and *move the entire world* in the opposite direction so that the scene appears as if viewed from the camera's position. The **view matrix** implements this inverse camera transform:

$$\mathbf{v}_{camera} = M_{view}\mathbf{v}_{world} \tag{74}$$

The view matrix is the inverse of the camera's world transform:

$$M_{view} = (T_{cam}R_{cam})^{-1} \tag{75}$$

For a camera at position $\mathbf{c}$ with rotation R_{cam}, the inverse can be computed in closed form. Because rotation matrices are orthogonal ($R^{-1} = R^T$):

$$M_{view} = \begin{bmatrix} R_{cam}^T & -R_{cam}^T\mathbf{c} \\ 0 & 1 \end{bmatrix} \tag{76}$$

The $-R_{cam}^T\mathbf{c}$ term translates the world so the camera ends up at the origin: it first undoes the camera's rotation, then translates by the negated (rotated) camera position.

Key properties:

- Camera is located at origin:

$$\mathbf{o}_{camera} = (0, 0, 0) \tag{77}$$

- Viewing direction along negative z-axis (a convention chosen so that z decreases with depth, which aligns naturally with a right-handed coordinate system)

4.4 Clip Space

Clip space is obtained by applying the **projection matrix**, which encodes the camera's field of view and depth range. The purpose of clip space is to bring all geometry that should be visible into a canonical volume, making it easy to test whether geometry is within the view frustum and to perform perspective division.

$$\mathbf{v}_{clip} = M_{proj}\mathbf{v}_{camera} \tag{78}$$

A standard **perspective projection matrix** is:

$$M_{proj} = \begin{bmatrix} \frac{1}{a\tan(\frac{\text{fov}}{2})} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan(\frac{\text{fov}}{2})} & 0 & 0 \\ 0 & 0 & \frac{f+n}{n-f} & \frac{2fn}{n-f} \\ 0 & 0 & -1 & 0 \end{bmatrix} \tag{79}$$

where:

$$a = \text{aspect ratio}, \quad n = \text{near plane}, \quad f = \text{far plane}, \quad \text{fov} = \text{field of view} \tag{80}$$

The -1 in the bottom row is key: it copies the negative z_{camera} value into the w component of the output. When we later divide by w (the perspective divide), points farther from the camera are divided by a larger value, making them appear smaller, this is precisely the perspective effect.

Vertices in clip space are 4D:

$$\mathbf{v}_{clip} = \begin{bmatrix} x_c \\ y_c \\ z_c \\ w_c \end{bmatrix} \tag{81}$$

Geometry that lies outside the view frustum satisfies $|x_c| > w_c$, $|y_c| > w_c$, or $|z_c| > w_c$, and is **clipped** (discarded or trimmed) at this stage.

4.5 Normalized Device Coordinates (NDC)

To obtain normalized device coordinates, the GPU performs the **perspective divide**, dividing each component by w_c:

$$\mathbf{v}_{ndc} = \begin{bmatrix} x_c/w_c \\ y_c/w_c \\ z_c/w_c \end{bmatrix} \tag{82}$$

This maps all visible geometry into a canonical cube:

$$x, y, z \in [-1, 1] \tag{83}$$

The perspective divide is what creates the visual perspective effect: a point at depth $z_{cam} = -2$ is divided by $w_c = 2$, halving its x and y extents and making it appear half as large as an equivalent point at depth $z_{cam} = -1$.

Key properties:

- Removes perspective distortion via division by w: NDC x and y are linearly related to horizontal and vertical screen position
- Enables uniform clipping and rasterization: all subsequent stages work in this device-independent coordinate system regardless of resolution or aspect ratio

4.6 Screen Space

Screen space maps normalized device coordinates to pixel coordinates on the display. This is the **viewport transform**, controlled by the application's viewport settings.

For a viewport of width W pixels and height H pixels:

$$x_{screen} = \frac{W}{2}(x_{ndc} + 1) \tag{84}$$

$$y_{screen} = \frac{H}{2}(1 - y_{ndc}) \tag{85}$$

$$z_{screen} = \frac{1}{2}(z_{ndc} + 1) \tag{86}$$

Note that the y formula inverts the axis: in NDC, $+y$ points upward, but screen coordinates traditionally have $+y$ pointing downward (row 0 at the top). The z value is remapped from $[-1, 1]$ to $[0, 1]$ for storage in the depth buffer.

Final screen coordinate:

$$\mathbf{v}_{screen} = \begin{bmatrix} x_{screen} \\ y_{screen} \\ z_{screen} \end{bmatrix} \tag{87}$$

4.7 Summary of Transform Pipeline

The full transformation pipeline is:

$$\mathbf{v}_{clip} = M_{proj} \cdot M_{view} \cdot M_{model} \cdot \mathbf{v}_{local} \tag{88}$$

Followed by the hardware-performed perspective divide:

$$\mathbf{v}_{ndc} = \frac{1}{w}\mathbf{v}_{clip} \tag{89}$$

And finally the viewport transform:

$$\mathbf{v}_{screen} = \text{Viewport}(\mathbf{v}_{ndc}) \tag{90}$$

4.8 Conceptual Interpretation

Each transformation corresponds to a change of coordinate system, each chosen so that the *next* stage's computation becomes simpler:

$$\text{Local} \rightarrow \text{World} \rightarrow \text{Camera} \rightarrow \text{Clip} \rightarrow \text{Screen} \tag{91}$$

- **Local**: convenient for modelling the mesh
- **World**: convenient for scene assembly and lighting
- **Camera**: convenient for perspective projection (camera at origin)

- **Clip/NDC**: convenient for clipping and rasterization (uniform $[-1, 1]$ cube)
- **Screen**: pixel coordinates for the rasterizer

The entire graphics pipeline can be viewed as a sequence of basis transformations, each applied to all vertex data, ultimately mapping 3D geometry to a 2D image on screen.

5 Rasterization

Rasterization is the process of converting geometric primitives (triangles) into discrete pixels on the screen. It is the defining algorithm of real-time graphics: rather than tracing rays from the camera (as in ray tracing), rasterization projects triangles forward onto the image plane and determines which pixels they cover.

Given vertices in screen space:

$$\mathbf{v}_0 = (x_0, y_0, z_0), \quad \mathbf{v}_1 = (x_1, y_1, z_1), \quad \mathbf{v}_2 = (x_2, y_2, z_2) \tag{92}$$

the goal is to determine which pixels lie inside the triangle and compute their attributes (colour, depth, texture coordinates) at each pixel.

5.1 Triangle Setup

The rasterizer works by iterating over all pixels in the bounding box of the triangle and testing each one for inclusion. Rather than solving the general point-in-triangle problem from scratch for each pixel, a set of **edge functions** is precomputed that allow this test to be performed with three comparisons and evaluated incrementally across scanlines.

A triangle is defined by three vertices in screen space:

$$\triangle = (\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2) \tag{93}$$

For a point $\mathbf{p} = (x, y)$, the edge function between vertices i and j is defined as:

$$E_{ij}(\mathbf{p}) = (x - x_i)(y_j - y_i) - (y - y_i)(x_j - x_i) \tag{94}$$

This is the z-component of the cross product of the edge vector $(\mathbf{v}_j - \mathbf{v}_i)$ with the vector from vertex i to $\mathbf{p}$. Its sign encodes which side of the directed edge $\overrightarrow{v_i v_j}$ the point lies on: positive on the left, negative on the right.

A point is inside a counter-clockwise wound triangle if and only if it is to the left of all three directed edges:

$$E_{01}(\mathbf{p}) \geq 0, \quad E_{12}(\mathbf{p}) \geq 0, \quad E_{20}(\mathbf{p}) \geq 0 \tag{95}$$

(for a consistently wound triangle)

Interpretation: Each edge function represents a signed area test. The value $E_{ij}(\mathbf{p})$ is twice the signed area of the triangle formed by $\mathbf{v}_i$, $\mathbf{v}_j$, and $\mathbf{p}$. All three being non-negative means $\mathbf{p}$ lies inside the triangle.

5.2 Barycentric Coordinates

Once we know a pixel centre $\mathbf{p}$ is inside the triangle, we need to interpolate per-vertex data. **Barycentric coordinates** provide exactly the right weights for this interpolation.

Any point inside the triangle can be expressed as a weighted combination of the vertices, with weights that sum to one:

$$\mathbf{p} = \alpha\mathbf{v}_0 + \beta\mathbf{v}_1 + \gamma\mathbf{v}_2 \tag{96}$$

with:

$$\alpha + \beta + \gamma = 1 \tag{97}$$

The weights have an elegant geometric interpretation in terms of areas. The barycentric coordinate α (associated with $\mathbf{v}_0$) equals the fraction of the total triangle area that the sub-triangle $(\mathbf{p}, \mathbf{v}_1, \mathbf{v}_2)$ occupies:

$$\alpha = \frac{A(\mathbf{p}, \mathbf{v}_1, \mathbf{v}_2)}{A(\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2)} \tag{98}$$

$$\beta = \frac{A(\mathbf{v}_0, \mathbf{p}, \mathbf{v}_2)}{A(\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2)} \tag{99}$$

$$\gamma = \frac{A(\mathbf{v}_0, \mathbf{v}_1, \mathbf{p})}{A(\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2)} \tag{100}$$

where the signed area of a triangle is:

$$A(\mathbf{a}, \mathbf{b}, \mathbf{c}) = \frac{1}{2}\begin{vmatrix} a_x & a_y & 1 \\ b_x & b_y & 1 \\ c_x & c_y & 1 \end{vmatrix} \tag{101}$$

Conveniently, the barycentric coordinates can be read directly from the edge functions: α, β, γ are proportional to $E_{12}(\mathbf{p})$, $E_{20}(\mathbf{p})$, $E_{01}(\mathbf{p})$ respectively, so the inside test and the coordinate computation share the same arithmetic.

5.3 Interpolation Across Surfaces

Once barycentric coordinates are known, any per-vertex attribute can be interpolated to obtain a smooth value at pixel $\mathbf{p}$.

Let q be a per-vertex attribute (e.g. depth, colour, normal, or texture coordinate):

$$q_0, \quad q_1, \quad q_2 \tag{102}$$

The interpolated value is the barycentric-weighted combination:

$$q(\mathbf{p}) = \alpha q_0 + \beta q_1 + \gamma q_2 \tag{103}$$

This gives a value that varies linearly across the triangle and matches each vertex value exactly at the corresponding corner.

Important: Perspective-Correct Interpolation Naively interpolating in screen space gives the wrong result when the triangle is viewed under perspective projection. The problem is that screen-space barycentric coordinates correspond to linear interpolation in 2D screen space, but the geometry was projected non-linearly: points farther from the camera were divided by a larger w. Equal steps in screen space do not correspond to equal steps along the 3D surface.

The fix is to interpolate the attribute divided by w (which *is* linear in screen space after the perspective divide), then undo the division:

$$q' = \frac{q}{w} \tag{104}$$

Interpolate q' using screen-space barycentric coordinates:

$$q'(\mathbf{p}) = \alpha q'_0 + \beta q'_1 + \gamma q'_2 \tag{105}$$

Also interpolate $\frac{1}{w}$:

$$\frac{1}{w(\mathbf{p})} = \alpha \frac{1}{w_0} + \beta \frac{1}{w_1} + \gamma \frac{1}{w_2} \tag{106}$$

Then recover the correct attribute value by dividing:

$$q(\mathbf{p}) = \frac{q'(\mathbf{p})}{\alpha \frac{1}{w_0} + \beta \frac{1}{w_1} + \gamma \frac{1}{w_2}} \tag{107}$$

Without this correction, texture mapping on a perspective-projected surface appears distorted: straight lines in texture space would appear as curves on screen. Modern GPUs perform this correction automatically.

5.4 Depth Testing (Z-Buffer)

Multiple triangles may project to the same pixel, but only the closest one should be visible. The **depth buffer** (or z-buffer) resolves this by storing the depth of the closest fragment seen so far at each pixel.

Each pixel stores a depth value:

$$z_{\text{buffer}}(x, y) \tag{108}$$

The buffer is initialised to the maximum depth (1.0) at the start of each frame. For each fragment produced by rasterization, with depth z_{new}:

$$z_{\text{new}} < z_{\text{buffer}}(x, y) \Rightarrow \text{fragment is visible} \tag{109}$$

If the test passes, the buffer and colour are updated:

$$z_{\text{buffer}}(x, y) \leftarrow z_{\text{new}} \tag{110}$$

$$\text{color}(x, y) \leftarrow \text{fragment color} \tag{111}$$

Otherwise, the fragment is discarded: a closer fragment has already been written here.

Depth Range: After the viewport transform, depth values are in:

$$z \in [0, 1] \tag{112}$$

Non-linearity: Depth is non-linear with respect to camera distance because of the perspective projection. The mapping from camera-space z to buffer depth is approximately:

$$z \sim \frac{1}{z_{camera}} \tag{113}$$

This means depth values are compressed for distant geometry and expanded for nearby geometry. As a result, more depth buffer precision is allocated near the camera (where small depth differences matter more) and less is allocated far away. However, placing the near plane very close to the camera compresses most geometry into a very small range of depth values near $z = 1$, causing **z-fighting** (flickering between two surfaces at similar depth).

5.5 Summary

Rasterization converts triangles into pixels through:

$$\text{Triangle} \rightarrow \text{Fragments} \rightarrow \text{Pixels} \tag{114}$$

Key steps:

- Determine pixel coverage using edge functions (inside-triangle test)
- Compute barycentric coordinates for each covered pixel
- Interpolate attributes (with perspective correction) across the surface
- Perform depth testing to resolve visibility between overlapping triangles

Rasterization is the bridge between the continuous mathematical description of a 3D scene and its discrete representation as a 2D array of pixels.

6 Lighting and Surface Shading

Rasterization tells us which pixel each surface fragment maps to, but not what colour it should be. Lighting models determine the apparent brightness and colour of a surface based on its orientation relative to light sources, the material properties of the surface, and the viewer's position. The simplest physically-motivated model is Lambertian (diffuse) shading.

Lighting determines how surfaces appear based on their orientation relative to light sources. The fundamental quantity governing lighting is the **surface normal**.

6.1 Surface Normals

A surface normal is a unit vector perpendicular to a surface. Intuitively, it encodes the orientation of the surface: a horizontal floor has a normal pointing straight up; a vertical wall has a normal pointing horizontally. Lighting is computed as a function of the angle between the normal and the light direction.

For a triangle with vertices $\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2$, two edge vectors lying in the plane of the triangle are:

$$\mathbf{e}_1 = \mathbf{v}_1 - \mathbf{v}_0 \tag{115}$$

$$\mathbf{e}_2 = \mathbf{v}_2 - \mathbf{v}_0 \tag{116}$$

The normal is their cross product, which is perpendicular to both edges and hence to the triangle plane:

$$\mathbf{n} = \mathbf{e}_1 \times \mathbf{e}_2 \tag{117}$$

Normalized to unit length for use in lighting calculations:

$$\hat{\mathbf{n}} = \frac{\mathbf{n}}{\|\mathbf{n}\|} \tag{118}$$

The direction of $\hat{\mathbf{n}}$ follows from the right-hand rule applied to the vertex winding order: for a counter-clockwise winding when viewed from outside, the normal points outward.

6.2 Light Direction

A **directional light** (such as the sun, which is so distant that all its rays are approximately parallel) is defined by a single unit vector:

$$\hat{\mathbf{l}} \in \mathbb{R}^3 \tag{119}$$

pointing *from the surface toward the light source* (not toward the surface, this sign convention ensures the dot product with the normal is positive for illuminated surfaces).

6.3 Diffuse Lighting (Lambertian Reflection)

The simplest and most physically-grounded lighting model is **Lambertian** (diffuse) reflection, which describes matte surfaces that scatter incoming light equally in all directions. The key physical insight is **Lambert's cosine law**: the amount of light striking a surface element depends on the cosine of the angle between the surface normal and the light direction.

Intuitively, when light hits a surface at a grazing angle (large θ), the same beam of light is spread over a larger area, reducing the irradiance (power per unit area) by a factor of $\cos\theta$. When light hits perpendicularly ($\theta = 0$), irradiance is maximised.

The Lambertian intensity is:

$$I = k_d \cdot \max(0, \hat{\mathbf{n}} \cdot \hat{\mathbf{l}}) \tag{120}$$

where:

- I is the reflected intensity (proportional to pixel brightness)
- $k_d \in [0, 1]$ is the **diffuse reflectance coefficient**, encoding how much light the material reflects (0 = completely black; 1 = perfectly white matte surface)

Geometric Interpretation: The dot product of two unit vectors equals the cosine of the angle between them:

$$\hat{\mathbf{n}} \cdot \hat{\mathbf{l}} = \cos\theta \tag{121}$$

Thus:

$$I \propto \cos\theta \tag{122}$$

Clamping:

$$\max(0, \cdot) \tag{123}$$

ensures that surfaces facing away from the light (where $\hat{\mathbf{n}} \cdot \hat{\mathbf{l}} < 0$, i.e. $\theta > 90$ř) receive no direct illumination. Without clamping, back-facing surfaces would be "negatively lit", which has no physical meaning.

6.4 Ambient Lighting

In reality, light bounces off multiple surfaces before reaching the eye, so even surfaces not directly facing a light source receive some indirect illumination. Accurately simulating this global illumination is expensive. **Ambient lighting** is a simple approximation: a constant, direction-independent base illumination added to every surface:

$$I_{ambient} = k_a \tag{124}$$

where k_a is a small constant chosen to ensure that shadowed surfaces are not completely black. It is a crude approximation, a uniform "fill light" with no geometric basis, but it prevents the extreme contrast that would result from purely Lambertian shading.

6.5 Combined Lighting Model

Adding ambient and diffuse contributions gives the **Lambertian shading model**:

$$I = k_a + k_d \cdot \max(0, \hat{\mathbf{n}} \cdot \hat{\mathbf{l}}) \tag{125}$$

This is the **Lambertian shading model**. It has only two parameters (k_a and k_d) and is extremely efficient to compute, making it suitable for real-time use. It accounts for the orientation of the surface relative to the light, but does not model specular highlights (shiny reflections), which require additional terms such as the Blinn-Phong or Cook-Torrance models.

6.6 Transforming Normals

When an object is transformed (moved, rotated, scaled), its normals must also be transformed, but *not* by the same matrix as the vertices. To see why, consider a non-uniform scale: if an object is stretched by a factor of 2 along x but not y, a 45° normal would point in a different direction relative to the now-elongated surface. Applying the same scale matrix to the normal would give the wrong result.

Given:

$$\mathbf{v}_{world} = M_{model}\mathbf{v}_{local} \tag{126}$$

The correct transform for normals is the **inverse transpose** of the model matrix:

$$\mathbf{n}_{world} = (M_{model}^{-1})^T \mathbf{n}_{local} \tag{127}$$

Reason: Normals must remain perpendicular to the surface after transformation. A tangent vector $\mathbf{t}$ lying on the surface transforms as $\mathbf{t}' = M\mathbf{t}$. For the transformed normal $\mathbf{n}'$ to be perpendicular to $\mathbf{t}'$, we need $\mathbf{n}' \cdot \mathbf{t}' = 0$. Substituting gives the requirement $(M^{-1})^T\mathbf{n}$. If M is a pure rotation or uniform scale, $(M^{-1})^T = M$ and no correction is needed, but for non-uniform scales or shears the correction is essential.

6.7 Basic Shadows (Visibility)

A point is in shadow if light cannot reach it because another object lies between it and the light source.

Define a **shadow ray** originating at surface point $\mathbf{p}$ in the direction of the light:

$$\mathbf{r}(t) = \mathbf{p} + t\hat{\mathbf{l}} \tag{128}$$

If this ray intersects another object before reaching the light ($t > 0$):

$$\exists t > 0 \quad \text{s.t.} \quad \mathbf{r}(t) \in \text{scene} \tag{129}$$

then the point is in shadow and receives only ambient illumination:

$$I = I_{ambient} \tag{130}$$

otherwise:

$$I = k_a + k_d \cdot \max(0, \hat{\mathbf{n}} \cdot \hat{\mathbf{l}}) \tag{131}$$

Casting shadow rays per fragment is the approach used in ray tracing. In rasterization, shadows are instead approximated by *shadow maps*.

6.8 Shadow Mapping (Rasterization Approximation)

In rasterization, shadows cannot be computed with shadow rays efficiently. Instead, shadows are approximated using a **shadow map**: a depth buffer rendered from the light's point of view.

Step 1: Render from Light's Perspective Render the scene from the light's viewpoint and store the depth of the nearest surface at each texel:

$$z_{light}(x, y) \tag{132}$$

This depth map records, for each direction from the light, the distance to the closest occluder.

Step 2: Compare Depth When shading a fragment at world position $\mathbf{p}$, transform $\mathbf{p}$ into the light's coordinate system and look up the stored depth. If the fragment is farther from the light than the stored depth, it is occluded:

$$z_{fragment} > z_{light} \quad \Rightarrow \quad \text{in shadow} \tag{133}$$

Shadow mapping is approximate: it introduces **shadow acne** (self-shadowing artifacts due to depth precision limits, corrected by adding a small bias) and **aliasing** (jagged shadow edges due to the finite resolution of the shadow map). Nevertheless, it is the dominant shadow technique in real-time graphics due to its simplicity and GPU-friendliness.

6.9 Interpolation of Normals

Per-vertex normals allow smooth shading of curved surfaces without subdividing the geometry. Instead of computing a single flat normal per triangle (which gives a faceted appearance), normals are stored per vertex representing the average surface orientation at that vertex, then interpolated across the triangle using barycentric coordinates:

$$\mathbf{n}(\mathbf{p}) = \alpha \mathbf{n}_0 + \beta \mathbf{n}_1 + \gamma \mathbf{n}_2 \tag{134}$$

Because the interpolated normal may no longer have unit length, it must be normalized before use in lighting:

$$\hat{\mathbf{n}}(\mathbf{p}) = \frac{\mathbf{n}(\mathbf{p})}{\|\mathbf{n}(\mathbf{p})\|} \tag{135}$$

This technique is called **Phong shading** (not to be confused with the Phong specular model). It produces smooth-looking shading with no extra geometric complexity. An even simpler approach, **Gouraud shading**, computes lighting at each vertex and interpolates the final colour, but this can miss sharp specular highlights that fall between vertices.

6.10 Summary

Lighting depends on:

$$\text{Surface Orientation} + \text{Light Direction} \tag{136}$$

Core computation:

$$I \propto \hat{\mathbf{n}} \cdot \hat{\mathbf{l}} \tag{137}$$

Thus, shading is fundamentally:

$$\text{Geometry} \rightarrow \text{Normals} \rightarrow \text{Dot Products} \rightarrow \text{Perceived Brightness} \tag{138}$$

7 Memory Layout of Graphics Data

All geometric data in a graphics pipeline is ultimately stored as structured sequences of bytes in memory (CPU RAM or GPU VRAM). These structures must be carefully designed for efficient access and parallel processing: GPUs achieve high throughput by reading large contiguous blocks of memory simultaneously across thousands of parallel threads, so how data is laid out has a direct impact on performance.

7.1 Vertex Buffers

A **vertex buffer** stores a sequence of vertices as a contiguous block of floats in memory:

$$V = [\mathbf{v}_0, \mathbf{v}_1, \ldots, \mathbf{v}_{n-1}] \tag{139}$$

Each vertex typically contains multiple **attributes**: the position, surface normal, texture coordinates, colour, tangent vector, etc. These are packed together into a struct per vertex:

$$\mathbf{v}_i = \Big[x_i, y_i, z_i, w_i,\ n_x, n_y, n_z,\ u, v,\ \ldots\Big] \tag{140}$$

Memory Layout: Each float attribute occupies 4 bytes. If there are k float components per vertex:

$$\text{size of vertex} = k \cdot 4 \text{ bytes} \tag{141}$$

This per-vertex size is called the **stride**: the number of bytes between the start of one vertex and the start of the next. The GPU uses the stride to step through the buffer when fetching vertex data.

Example (Position Only):

$$\mathbf{v}_i = (x, y, z), \quad k = 3 \tag{142}$$

$$\text{vertex size} = 3 \times 4 = 12 \text{ bytes} \tag{143}$$

Contiguous Storage: All vertices are stored one after another with no gaps:

$$\text{Vertex Buffer} = [x_0, y_0, z_0,\ x_1, y_1, z_1,\ \ldots] \tag{144}$$

This **interleaved** layout (all attributes for vertex i stored together before vertex $i+1$) is the most common arrangement because it maximises spatial locality: when the GPU fetches a vertex, all its attributes are in the same cache line. An alternative layout, storing all positions together, then all normals together, is called **non-interleaved**, and can be beneficial when different shader stages access different subsets of attributes.

7.2 Index Buffers

An **index buffer** defines how vertices are connected into triangles. Rather than storing vertex data repeatedly, the index buffer stores integer indices into the vertex buffer:

$$F = [(i_0, i_1, i_2), (i_3, i_4, i_5), \dots] \tag{145}$$

Each triplet of indices defines one triangle:

$$\triangle_k = (\mathbf{v}_{i_a}, \mathbf{v}_{i_b}, \mathbf{v}_{i_c}) \tag{146}$$

Memory Representation: Indices are stored as integers, not floats. The bit width is chosen based on the number of vertices:

$$\text{index size} = \begin{cases} 2 \text{ bytes (16-bit)} & \text{if } n_v \leq 65536 \\ 4 \text{ bytes (32-bit)} & \text{if } n_v > 65536 \end{cases} \tag{147}$$

Smaller indices save memory and improve cache utilisation. Most meshes in games have fewer than 2^{16} vertices, making 16-bit indices the common choice.

Example:

$$F = [0, 1, 2,\ 2, 3, 0] \tag{148}$$

This represents two triangles sharing vertices 0 and 2, forming a quad. Vertex 0 appears in both triangles but is stored only once in the vertex buffer.

7.3 Transformation Storage

Each object in a scene has an associated transformation matrix that places it in the world:

$$M \in \mathbb{R}^{4\times 4} \tag{149}$$

Matrix Layout: A 4×4 matrix has 16 scalar entries:

$$M = \begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \\ m_{41} & m_{42} & m_{43} & m_{44} \end{bmatrix} \tag{150}$$

Memory Size: Each entry is a float (4 bytes), so:

$$\text{matrix size} = 16 \times 4 = 64 \text{ bytes} \tag{151}$$

Storage Format: The matrix is stored as a flat array of 16 floats:

$$[m_{11}, m_{12}, \dots, m_{44}] \tag{152}$$

Note on Row vs Column Major: Different APIs and languages store matrices differently in memory:

- **Row-major**: row 0 is stored first (C/C++ arrays, HLSL by default). Order: $m_{11}, m_{12}, m_{13}, m_{14}, m_{21}, \ldots$
- **Column-major**: column 0 is stored first (OpenGL/GLSL convention, MATLAB). Order: $m_{11}, m_{21}, m_{31}, m_{41}, m_{12}, \ldots$

A matrix that looks correct on paper may produce wrong results at runtime if the expected memory layout does not match how the GPU reads it. This is a common source of bugs when mixing APIs or libraries with different conventions.

7.4 Estimating Memory Usage

Let:

$$n_v = \text{number of vertices} \tag{153}$$

$$n_t = \text{number of triangles} \tag{154}$$

$$k = \text{floats per vertex} \tag{155}$$

Vertex Buffer Size:

$$\text{size}_V = n_v \cdot k \cdot 4 \tag{156}$$

Index Buffer Size: Each triangle uses 3 indices, each of size s_i bytes:

$$\text{size}_F = n_t \cdot 3 \cdot s_i \tag{157}$$

where:

$$s_i \in \{2, 4\} \text{ bytes} \tag{158}$$

Transformation Size: For n_o objects, each requiring one 4×4 matrix:

$$\text{size}_M = n_o \cdot 64 \tag{159}$$

Total Memory:

$$\text{Total} = \text{size}_V + \text{size}_F + \text{size}_M \tag{160}$$

7.5 Example: Pyramid

Consider a pyramid with:

- $n_v = 5$ vertices (4 base corners + 1 apex)
- $n_t = 6$ triangles (4 side faces + 2 triangles for the square base)
- $k = 3$ (position only: x, y, z)

Vertex Memory:

$$\text{size}_V = 5 \cdot 3 \cdot 4 = 60 \text{ bytes} \tag{161}$$

Index Memory (32-bit):

$$\text{size}_F = 6 \cdot 3 \cdot 4 = 72 \text{ bytes} \tag{162}$$

Transform:

$$\text{size}_M = 64 \text{ bytes} \tag{163}$$

Total:

$$\text{Total} = 60 + 72 + 64 = 196 \text{ bytes} \tag{164}$$

For context, a modern high-resolution game mesh with position, normal, tangent, and UV ($k = 11$) and $n_v = 100{,}000$ vertices would consume around 4.4 MB of vertex data alone.

7.6 Summary

Graphics data in memory is structured as a collection of per-object tuples:

$$\text{Scene} = \{(V_i, F_i, M_i)\}_{i=1}^{n_o} \tag{165}$$

Each component is stored as contiguous bytes:

$$\text{Bytes} \to \text{Floats} \to \text{Vertices} \to \text{Triangles} \to \text{Objects} \tag{166}$$

Efficient layout enables:

- **Coalesced memory access on GPU**: when many parallel threads read consecutive addresses simultaneously, the hardware satisfies the requests in a single memory transaction rather than many separate ones
- **Parallel processing of vertices**: each vertex is an independent unit of work that can be processed by a different GPU thread simultaneously
- **High throughput rendering**: well-structured data minimises the time GPUs spend waiting for memory, keeping the arithmetic units busy

8 Specialized Hardware in GPUs

Modern GPUs are highly specialized processors designed to execute large numbers of similar operations in parallel. Over time, they have evolved from fixed-function graphics devices to programmable, massively parallel compute units with dedicated hardware for specific tasks. Understanding what is handled in hardware versus software is essential for writing efficient graphics code.

8.1 Graphics Pipelines and Fixed Function Units

Early GPUs (circa 1990s) implemented a **fixed-function pipeline**: every stage of the rendering process was hardwired in silicon and could only be configured via a limited set of parameters. The programmer could not write custom shading code; they could only set state variables (lighting colours, texture modes, etc.) and the hardware performed a predetermined computation.

The classical pipeline can be represented as:

$$\mathbf{v}_{clip} = M_{proj} \cdot M_{view} \cdot M_{model} \cdot \mathbf{v}_{local} \tag{167}$$

followed by rasterization:

$$\text{Triangles} \to \text{Fragments} \to \text{Pixels} \tag{168}$$

Fixed-Function Stages:

- Vertex transformation (multiply by $M_{proj}M_{view}M_{model}$)
- Primitive assembly (group vertices into triangles)
- Clipping (discard geometry outside the view frustum)
- Rasterization (determine which pixels each triangle covers)
- Depth testing (z-buffer visibility resolution)

These stages were implemented directly in hardware, meaning:

$$\text{Transformation} = \text{Dedicated combinational logic} \tag{169}$$

The advantage was speed and power efficiency; the disadvantage was inflexibility, every application was constrained to the same hardware lighting equations and texture blending modes.

Modern GPUs replace most fixed-function stages with **programmable shaders** written in languages like GLSL or HLSL:

$$\text{Vertex Shader}, \quad \text{Fragment Shader}, \quad \text{Compute Shader} \tag{170}$$

However, some stages (rasterization and depth testing) remain implemented in fixed hardware, as they are simple, well-defined operations executed billions of times per second that benefit enormously from dedicated silicon. Programmable shaders execute on general-purpose shader processors that share cores and run in a **SIMT** (Single Instruction, Multiple Threads) fashion: a group of threads (called a *warp* or *wavefront*) executes the same instruction simultaneously on different data. This is why branching in shaders can be expensive, divergent branches cause some threads to idle while others execute.

8.2 Ray Tracing Cores (RT Cores)

Ray tracing computes visibility and lighting by casting rays through the scene and testing for intersections with geometry. Unlike rasterization, which projects geometry *forward* onto the screen, ray tracing traces rays *backward* from the camera into the scene.

A ray is defined as:

$$\mathbf{r}(t) = \mathbf{o} + t\mathbf{d} \tag{171}$$

where:

- $\mathbf{o}$ is the ray origin (the camera position for primary rays)
- $\mathbf{d}$ is the unit direction vector
- $t \geq 0$ parameterises the ray; increasing t moves further along the ray

The core operation is ray-geometry intersection:

$$\mathbf{r}(t) = \text{surface} \tag{172}$$

For triangles, this involves solving:

$$\mathbf{o} + t\mathbf{d} = \alpha\mathbf{v}_0 + \beta\mathbf{v}_1 + \gamma\mathbf{v}_2 \tag{173}$$

subject to:

$$\alpha + \beta + \gamma = 1 \tag{174}$$

Acceleration Structures: Testing a ray against every triangle in a scene is $O(n)$ per ray. With millions of triangles and millions of rays, this is prohibitively expensive. Instead, triangles are organized into a **Bounding Volume Hierarchy (BVH)**: a tree where each internal node stores an axis-aligned bounding box (AABB) enclosing all the triangles in its subtree. Ray traversal starts at the root, recursively descending only into children whose bounding boxes the ray intersects. This reduces the expected traversal cost from:

$$O(n) \to O(\log n) \tag{175}$$

for scene traversal.

RT cores accelerate the two most expensive parts of ray tracing in hardware:

- **Ray-AABB intersection**: testing a ray against a bounding box for BVH traversal
- **Ray-triangle intersection**: the precise intersection test once a leaf node is reached

These tests are performed by fixed-function hardware units running in parallel with the shader cores, freeing the shader processors to handle shading while the RT cores handle traversal.

8.3 Tensor Cores and Machine Learning

Tensor cores are specialized units designed specifically to accelerate **matrix multiplication**, the fundamental operation in deep learning and linear algebra.

The fundamental operation is matrix multiplication:

$$C = A \cdot B \tag{176}$$

with:

$$C_{ij} = \sum_k A_{ik} B_{kj} \tag{177}$$

These operations are central to:

- Neural networks (matrix-multiply-heavy forward and backward passes)
- Linear algebra (solving systems, decompositions)
- Physics simulations (finite element methods)

Tensor cores perform **fused multiply-add (FMA)** operations on entire small matrix tiles in a single instruction:

$$D = A \cdot B + C \tag{178}$$

The "fused" part means the multiply and add are performed in a single step without intermediate rounding, which improves both speed and numerical accuracy.

Key Property: Tensor cores can perform this operation in substantially fewer clock cycles than general shader cores, because the hardware is physically wired to implement the specific pattern of multiplications and additions required by the matrix product:

$$\text{Throughput} \gg \text{CPU scalar performance} \tag{179}$$

Modern applications include **AI upscaling** (e.g. DLSS): a neural network runs on tensor cores to upscale a low-resolution rendered frame to full resolution, producing a result visually similar to native resolution at a fraction of the rendering cost.

8.4 Evolution of GPU Specialization

The evolution of GPUs can be understood as a progression from a single dedicated purpose (rendering fixed-pipeline graphics) toward increasingly general and specialized parallel computation.

Stage 1: Fixed-Function Graphics (1990s)

$$\text{Hardwired pipeline for rendering} \tag{180}$$

All rendering operations encoded directly in silicon. Fast and efficient, but inflexible: programmers could not write custom shading code.

Stage 2: Programmable Shaders (early 2000s)

$$\text{Generalized arithmetic units (ALUs)} \tag{181}$$

Vertex and pixel shaders replaced fixed transformation and lighting. Programmers could write custom programs running on the GPU, unlocking per-pixel lighting, environment mapping, and procedural effects.

Stage 3: General-Purpose GPU (GPGPU, mid 2000s)

$$\text{Parallel execution of arbitrary compute kernels} \tag{182}$$

APIs like CUDA and OpenCL exposed the GPU's parallel compute power for non-graphics tasks: physics simulation, machine learning, scientific computing. The GPU became a general parallel processor.

Stage 4: Specialized Accelerators (present)

- RT cores (ray tracing)
- Tensor cores (matrix operations)

$$\text{Specialized hardware} \Rightarrow \text{higher efficiency per watt} \tag{183}$$

Each specialization trades flexibility for throughput: a general ALU can compute anything but does it relatively slowly; a dedicated unit does one thing extremely fast. The art of GPU architecture is determining which operations are frequent and regular enough to justify dedicated silicon.

8.5 Conceptual Interpretation

All GPU operations ultimately reduce to:

$$\text{Logic Gates} \rightarrow \text{Arithmetic Units} \rightarrow \text{Matrix Operations} \tag{184}$$

Thus:

$$\text{Graphics} \approx \text{Linear Algebra} \approx \text{Parallel Computation} \tag{185}$$

The GPU is therefore best understood as:

$$\text{A massively parallel linear algebra processor with specialized accelerators} \tag{186}$$

The entire 3D graphics pipeline, from raw bits in memory to pixels on screen, is a sequence of structured linear algebra operations, executed in parallel across thousands of cores, with specialized hardware handling the most repetitive and performance-critical steps.

9 Shaders in Practice

The previous sections described the 3D graphics pipeline in abstract mathematical terms: vertices transformed through coordinate spaces, triangles rasterized into fragments, and fragments shaded with lighting models. In practice, these computations are programmed explicitly by the developer in a **shader**: a small program that runs on the GPU for every vertex or every fragment.

This section grounds the abstract pipeline in a concrete setting: the Unity game engine, using its **ShaderLab** file format. ShaderLab is Unity's shader language wrapper, providing a declarative structure (properties, subshaders, passes) that organises how data flows into the GPU. Inside that wrapper, the actual vertex and fragment computations are written in **HLSL** (High-Level Shading Language), a C-like language that compiles to GPU machine code.

The central question this section answers is: how does geometry stored in memory as bytes become a coloured pixel on screen? The answer is a chain of explicitly programmed and hardware-fixed stages, all orchestrated by the shader file:

$$\text{Mesh+Material} \rightarrow \text{Vertex Shader (HLSL)} \rightarrow \text{Rasterizer (hardware)} \rightarrow \text{Fragment Shader (HLSL)} \rightarrow \text{Fram} \tag{187}$$

9.1 The ShaderLab File Structure

A Unity shader is a single `.shader` text file written in **ShaderLab**. It defines everything the GPU needs to render a surface: what configurable parameters it exposes, how geometry should be rendered, and what HLSL programs execute at each stage.

The top-level structure is:

```
Shader "MyShaders/Example"
{
    Properties { ... }

    SubShader
    {
        Tags { ... }

        Pass
        {
            HLSLPROGRAM
            // vertex and fragment shader HLSL code
            ENDHLSL
        }
    }
}
```

The three structural blocks serve distinct roles:

- **Properties**: declares the shader's configurable inputs, colours, textures, scalar values, that can be set per-material in the editor or from script
- **SubShader**: defines a complete rendering strategy. A shader file may contain multiple SubShaders as fallbacks for different hardware; Unity selects the first one the GPU supports
- **Pass**: contains one draw call's GPU programs. A SubShader can have multiple passes (for example, one pass for geometry and another for shadow casting). Each pass contains the HLSL vertex and fragment shaders

9.2 Properties: Data Passed Into the Shader

A shader receives data from two sources: the **mesh** (per-vertex geometry data stored in the vertex buffer) and the **material** (per-object constants set by the artist or programmer). These correspond to two fundamentally different memory access patterns on the GPU.

Mesh Data (Per-Vertex Attributes): Each vertex in the mesh's vertex buffer carries a set of attributes. These are uploaded to the GPU once and read by the vertex shader independently for each vertex. The standard per-vertex attributes are:

- **Position** $\mathbf{v}_{local} \in \mathbb{R}^3$: the vertex location in local object space
- **Normal** $\hat{\mathbf{n}} \in \mathbb{R}^3$: unit surface normal at the vertex, used for lighting computations
- **Tangent** $\hat{\mathbf{t}} \in \mathbb{R}^4$: a vector tangent to the surface; the fourth component encodes handedness (the sign of the bitangent), required for normal mapping
- **UV coordinates** $(u, v) \in [0, 1]^2$: texture sampling coordinates that map the surface to a 2D texture image
- **Vertex colour** $(r, g, b, a) \in [0, 1]^4$: per-vertex colour data, used for artistic effects or baked lighting

These are exactly the attributes packed into the interleaved vertex buffer described in Section 8.1. The GPU uses the stride to step through the buffer and delivers the correct bytes for each vertex at the start of the vertex shader.

Material Data (Uniform Constants): Material data is declared in the `Properties` block and uploaded to the GPU as **uniform variables**: values that are constant across all vertices and fragments within a single draw call. They are written once on the CPU side and read many times on the GPU.

```
Properties
{
    _Color        ("Base Colour",  Color)        = (1, 1, 1, 1)
    _MainTex      ("Albedo Texture", 2D)         = "white" {}
    _Glossiness   ("Smoothness",  Range(0, 1))   = 0.5
    _Metallic     ("Metallic",    Range(0, 1))   = 0.0
}
```

The supported property types are:

- `Color`: a four-component RGBA value, passed to the shader as a `float4`
- `2D`: a 2D texture; the GPU reads texels from it when the shader calls a sample instruction
- `Float` / `Range`: a scalar value; `Range(a, b)` additionally clamps the editor slider to $[a, b]$
- `Vector`: a four-component `float4` for arbitrary vector data

The two data sources map directly onto the pipeline structures developed in earlier sections:

$$\text{Vertex buffer (VBO)} \rightarrow \text{per-vertex: position, normal, tangent, UV, colour} \tag{188}$$

$$\text{Uniform buffer} \rightarrow \text{per-draw-call: colour, texture, smoothness, } \ldots \tag{189}$$

9.3 The SubShader and Pass

A **SubShader** groups passes under shared render state. **Tags** communicate metadata to Unity's render pipeline:

```
SubShader
{
    Tags { "RenderType" = "Opaque"  "Queue" = "Geometry" }
    ...
}
```

`RenderType` identifies how the surface should be treated (opaque, transparent, etc.), and `Queue` controls draw order: opaque geometry is drawn before transparent geometry so that the depth buffer resolves visibility correctly.

A **Pass** configures the fixed-function GPU state and contains the HLSL programs. The render state directives mirror the fixed-function stages described in earlier sections:

```
Pass
{
    Cull  Back
    ZTest LEqual
    ZWrite On
    Blend Off

    HLSLPROGRAM
    #pragma vertex   vert
    #pragma fragment frag
    // ... HLSL code ...
    ENDHLSL
}
```

- `Cull Back`: discard triangles whose winding order indicates they face away from the camera (back-face culling, Section 2.3)
- `ZTest LEqual`: pass fragments whose depth is less than or equal to the stored depth buffer value (Section 5.4)
- `ZWrite On`: write the fragment's depth to the depth buffer when the test passes
- `Blend Off`: disable alpha blending; the fragment colour overwrites the framebuffer directly

The `#pragma` directives tell the HLSL compiler which functions are the vertex and fragment entry points.

9.4 The Vertex Shader (HLSL)

The **vertex shader** is an HLSL function that runs once per vertex. It receives the per-vertex attributes from the vertex buffer and must output a clip-space position. It may also compute additional data, world-space normals, interpolated UVs, to pass down to the fragment shader.

Input Struct (Attributes): Vertex shader inputs are declared as an HLSL struct. Each field is annotated with a **semantic**: a keyword that tells the GPU driver which vertex buffer attribute to bind to that field:

```
struct Attributes
{
    float4 positionOS : POSITION;   // local-space position (w = 1)
```

```
    float3 normalOS   : NORMAL;     // local-space surface normal
    float4 tangentOS  : TANGENT;    // local-space tangent (w =
       handedness)
    float2 uv         : TEXCOORD0;  // primary UV coordinates
    float4 color      : COLOR;      // per-vertex colour
};
```

Output Struct (Varyings): The output struct carries values from the vertex shader through the rasterizer to the fragment shader. These are called **varyings** because the rasterizer interpolates them across the triangle surface for each fragment:

```
struct Varyings
{
    float4 positionCS : SV_POSITION; // clip-space position (required)
    float3 normalWS   : TEXCOORD0;   // world-space normal for lighting
    float2 uv         : TEXCOORD1;   // UV for texture sampling
    float4 color      : COLOR;       // interpolated vertex colour
};
```

The semantic `SV_POSITION` (System Value Position) is mandatory: it is the clip-space vertex position used by the rasterizer to determine triangle coverage.

Vertex Shader Entry Point: The `vert` function applies the model–view–projection transform and prepares the interpolants:

```
Varyings vert(Attributes IN)
{
    Varyings OUT;

    // Transform position: local -> world -> clip space
    float4 worldPos = mul(UNITY_MATRIX_M,  IN.positionOS);
    OUT.positionCS  = mul(UNITY_MATRIX_VP, worldPos);

    // Transform normal using the inverse-transpose of the model matrix
    // (Section 6.5): preserves perpendicularity under non-uniform
       scale
    float3x3 normalMatrix = transpose((float3x3)UNITY_MATRIX_I_M);
    OUT.normalWS = normalize(mul(normalMatrix, IN.normalOS));

    // UV coordinates and vertex colour pass through unchanged
    OUT.uv    = IN.uv;
    OUT.color = IN.color;

    return OUT;
}
```

Unity supplies built-in matrix uniforms:

$$\texttt{UNITY_MATRIX_M} = M_{model}, \quad \texttt{UNITY_MATRIX_VP} = M_{proj} \cdot M_{view}, \quad \texttt{UNITY_MATRIX_I_M} = M_{model}^{-1} \tag{190}$$

so the combined position transform is exactly:

$$\mathbf{v}_{clip} = M_{proj} \cdot M_{view} \cdot M_{model} \cdot \mathbf{v}_{local} \tag{191}$$

as derived in Section 4. Each vertex is processed independently, making the vertex shader trivially parallelisable across the GPU's thousands of shader cores.

9.5 Rasterization (Fixed Hardware Stage)

After the vertex shader has run on all three vertices of a triangle, the GPU's fixed-function **rasterizer** takes over. No HLSL is written for this stage; it is implemented entirely in dedicated silicon.

Given the three clip-space positions from the vertex shader:

$$\mathbf{p}_{clip}^{(0)}, \quad \mathbf{p}_{clip}^{(1)}, \quad \mathbf{p}_{clip}^{(2)} \tag{192}$$

the rasterizer performs the following steps automatically:

- **Perspective divide and viewport transform**: converts clip-space coordinates to screen-space pixel positions (Sections 4.5–4.6)
- **Triangle coverage**: determines which pixel centres lie inside the triangle using edge functions (Section 5.1)
- **Barycentric interpolation**: computes (α, β, γ) for each covered pixel and uses them to interpolate all varyings (normals, UVs, colours) with perspective-correct weights (Sections 5.2–5.3)
- **Depth test**: compares the interpolated depth against the depth buffer and discards occluded fragments (Section 5.4)

Each surviving pixel produces a **fragment**: an interpolated set of varyings together with a screen-space position, ready for the fragment shader. The rasterizer bridges the programmed vertex stage and the programmed fragment stage without any developer code.

9.6 The Fragment Shader (HLSL)

The **fragment shader** is an HLSL function that runs once per fragment. It receives the rasterizer's interpolated varyings and computes the final colour written to the framebuffer.

Fragment Shader Entry Point:

```
float4 frag(Varyings IN) : SV_Target
{
    // Sample the albedo texture at the interpolated UV coordinates
    float4 albedo = tex2D(_MainTex, IN.uv) * _Color;

    // Re-normalise the surface normal (interpolation can reduce its
       length)
    float3 N = normalize(IN.normalWS);

    // Directional light direction (unit vector toward the light source
       )
    float3 L = normalize(_WorldSpaceLightPos0.xyz);

    // Lambertian diffuse term: I proportional to cos(theta) (Section
       6.3)
    float diff = max(0.0, dot(N, L));

    // Combine ambient and diffuse contributions modulated by albedo
       colour
    float3 color = albedo.rgb * (_AmbientColor.rgb + diff);

    return float4(color, albedo.a);
}
```

The semantic `SV_Target` marks the return value as the colour to be written to the current render target. The fragment shader has access to three categories of input:

- **Interpolated varyings**: per-fragment values produced by the rasterizer from the vertex shader outputs (normals, UVs, colours)
- **Material uniforms**: constants declared in the Properties block (`_MainTex`, `_Color`, etc.), constant across the draw call
- **Engine uniforms**: values supplied automatically by Unity, such as `_WorldSpaceLightPos0` (directional light direction) and `_AmbientColor`

The lighting computation implements the Lambertian shading model derived in Section 6:

$$I = k_a + k_d \cdot \max(0,\, \hat{\mathbf{n}} \cdot \hat{\mathbf{l}}) \tag{193}$$

where k_d is the albedo colour sampled from the texture and $\hat{\mathbf{n}}$ is the per-fragment interpolated world-space normal. The result is a `float4` RGBA colour that is written to the framebuffer at the pixel's screen-space coordinates.

9.7 Swizzling

HLSL vectors have named components that can be accessed and recombined freely using **swizzle notation**, a compact syntax for selecting, reordering, or repeating the scalar elements of a vector in a single expression.

Component names. Each component of a vector can be referred to by either of two equivalent sets of names:

- `.xyzw`, positional convention, used when the vector represents a spatial quantity (position, direction, normal)
- `.rgba`, colour convention, used when the vector represents a colour value

Both sets address the same underlying components in order; `.x` and `.r` both refer to the first component. The two sets cannot be mixed within a single swizzle expression.

Reading components. Any subset of components can be extracted to form a shorter vector:

```
float4 col = float4(0.8, 0.4, 0.2, 1.0);

float3 rgb  = col.rgb;   // (0.8, 0.4, 0.2)  -- drop alpha
float2 rg   = col.rg;    // (0.8, 0.4)
float  a    = col.a;     // 1.0               -- scalar extraction

float3 pos  = col.xyz;   // same components, positional names
float  z    = col.z;     // 0.2
```

Reordering and repeating components. Components can appear in any order and may be repeated:

```
float4 col = float4(0.8, 0.4, 0.2, 1.0);

float4 rev  = col.abgr;  // (1.0, 0.2, 0.4, 0.8)  -- reversed
float3 grb  = col.grb;   // (0.4, 0.8, 0.2)        -- reordered
float3 rrr  = col.rrr;   // (0.8, 0.8, 0.8)        -- repeated (
   greyscale)
float2 aa   = col.aa;    // (1.0, 1.0)
```

Writing via swizzle. Swizzle notation is also valid on the left-hand side of an assignment, provided no component is written more than once:

```
float4 col = float4(0.0, 0.0, 0.0, 1.0);
col.rgb = float3(0.8, 0.4, 0.2);  // sets r, g, b; leaves a unchanged
col.xy  = col.yx;                 // swap x and y in place
```

Common patterns in Unity shaders. Swizzling appears throughout fragment shader code. Typical uses include:

```
// Extract colour channels from a texture sample (float4 -> float3)
float4 sample = tex2D(_MainTex, IN.uv);
float3 albedo  = sample.rgb;
float  alpha   = sample.a;

// Construct a float4 from a float3 and a literal alpha
float3 litColor = albedo * diff;
return float4(litColor, 1.0);

// Re-normalise a float4 position to float3 for arithmetic
float3 worldPos = IN.positionWS.xyz;
```

Swizzling has zero runtime cost, the GPU compiler resolves component selection at compile time, mapping each swizzle directly onto register lanes in the SIMD execution units.

9.8 Data Flow Summary

The complete journey from stored data to pixel colour is a sequence of transformations, each stage consuming the output of the previous:

$$\text{Mesh (VBO + IBO)} \xrightarrow{\text{vertex fetch}} \text{Attributes} \xrightarrow{\texttt{vert()}} \text{Varyings} \xrightarrow{\text{rasterizer}} \text{Fragments} \xrightarrow{\texttt{frag()}} \text{Pixel colour} \tag{194}$$

Data enters the pipeline from two sources at two different granularities:

$$\begin{cases} \text{Mesh (vertex buffer)} & \to \text{per-vertex: position, normal, tangent, UV, colour} \\ \text{Material (uniform buffer)} & \to \text{per-draw-call: textures, colours, scalar parameters} \end{cases} \tag{195}$$

The ShaderLab file is the complete specification of this computation:

$$\text{ShaderLab} = \underbrace{\text{Properties}}_{\text{data interface}} + \underbrace{\text{Pass state}}_{\text{fixed-function config}} + \underbrace{\text{HLSL programs}}_{\text{programmable stages}} \tag{196}$$

The responsibilities of each stage are:

- **Vertex shader (HLSL)**: transforms each vertex from local space to clip space; prepares interpolated data to carry to the fragment shader
- **Rasterizer (fixed hardware)**: determines pixel coverage; interpolates varyings across the triangle with perspective correction; performs depth testing
- **Fragment shader (HLSL)**: samples textures; evaluates lighting from interpolated normals and material uniforms; outputs a final RGBA colour

Thus, the mathematical pipeline developed throughout these notes, from bits in memory (Section 1), through geometry (Section 2), coordinate transforms (Sections 3–4), rasterization (Section 5), and lighting (Section 6), maps directly and completely onto the structure of a single ShaderLab file, executable on real GPU hardware.

$$\text{Bits} \to \text{Vertices} \to \text{Clip Space} \to \text{Fragments} \to \text{Shading} \to \text{Pixels} \tag{197}$$

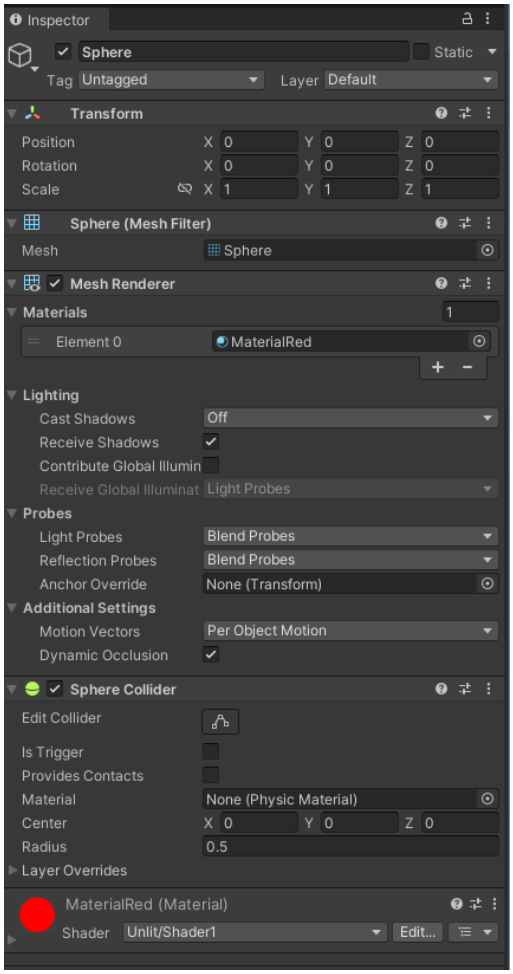
10 Unity: Unlit Shaders

An **unlit shader** computes a colour for every fragment without consulting the scene's lights, shadows, or surface normals. There is no illumination model. Every fragment belonging to the surface returns the same value, determined entirely by the material's uniform parameters rather than any property of the geometry or the scene. This absence of lighting complexity makes unlit shaders the clearest possible lens through which to examine the shader pipeline in isolation: the full vertex-to-fragment journey still executes, but the fragment stage reduces to a trivial constant function, making it easy to see exactly where each piece of data comes from and where it goes.

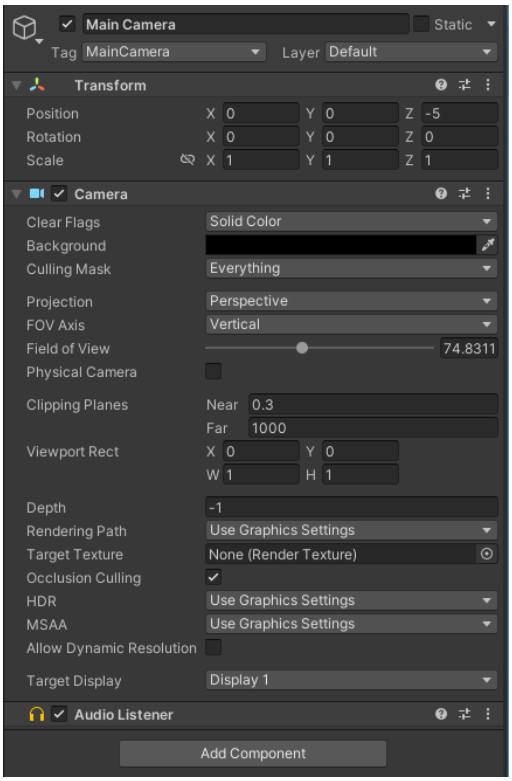
10.1 Basic Colour

The most elementary unlit shader outputs a single, solid colour across the entire surface. There is one material property, one line of vertex computation, and one line of fragment computation. The geometry is transformed correctly, rasterized correctly, and coverage-tested correctly, yet the fragment shader discards every interpolated attribute and returns a constant.

Scene setup. The scene contains two objects: a Unity primitive sphere and the main camera (Figures 1a and 1b).



(a) Sphere inspector. The object sits at the origin with default scale. The `Shader1` material is applied, with the `_Color` property set to red: $(r, g, b, a) = (1, 0, 0, 1)$.



(b) Camera inspector. The camera is placed at $(0, 0, -5)$ in world space, aimed along $+z$ toward the origin. Perspective projection, FOV $= 74.83$ř, near $= 0.3$, far $= 1000$. The background is solid black.

Figure 1: The complete scene: a sphere at the origin and a camera looking at it from $z = -5$.

The camera's view frustum is therefore aligned with $-z$ pointing into the scene. The sphere sits at the origin, which in view space maps to $(0, 0, 5)$, squarely in front of the near and far clipping planes. The coordinate transform chain from Section 4 applies without modification:

$$\mathbf{v}_{clip} = M_{proj} \cdot M_{view} \cdot M_{model} \cdot \mathbf{v}_{local} \tag{198}$$

where $M_{model} = I$ (the sphere is at the origin, unrotated, unit scale) and M_{view} encodes the camera translation of -5 along z.

The shader. The complete `Shader1.shader` file is:

```
Shader "Unlit/Shader1"
{
    Properties
    {
        _Color ("Color", Color) = (1, 1, 1, 1)
    }

    SubShader
```

```
    {
        Tags { "RenderType"="Opaque" }

        Pass
        {
            CGPROGRAM
            #pragma vertex vert
            #pragma fragment frag

            #include "UnityCG.cginc"

            float4 _Color;

            struct MeshData
            {
                float4 vertex : POSITION;
            };

            struct Interpolators
            {
                float4 vertex : SV_POSITION;
            };

            Interpolators vert(MeshData v)
            {
                Interpolators o;
                o.vertex = UnityObjectToClipPos(v.vertex);
                return o;
            }

            float4 frag(Interpolators i) : SV_Target
            {
                return _Color;
            }
            ENDCG
        }
    }
}
```

Each block has a specific and minimal role.

CGPROGRAM and UnityCG.cginc. This shader uses `CGPROGRAM`/`ENDCG` rather than the `HLSLPROGRAM`/`ENDHLSL` blocks used in the theoretical sections of these notes. Both compile HLSL; the difference is the include path. `CGPROGRAM` pulls in Unity's legacy `UnityCG.cginc` header, which is standard for the Built-in Render Pipeline and provides convenience macros and functions such as `UnityObjectToClipPos`. The underlying language and GPU execution model are identical.

Properties block.

```
_Color ("Color", Color) = (1, 1, 1, 1)
```

A single `Color` uniform is declared, defaulting to opaque white. It is exposed in the Material inspector, where it has been set to red $(1, 0, 0, 1)$ for this render. Inside the `CGPROGRAM` block the matching HLSL variable:

```
float4 _Color;
```

receives the inspector value. This is the per-draw-call uniform path described in Section 9.2: the CPU writes the value once into a constant buffer and the GPU reads it identically in every fragment invocation.

Vertex input and output structs.

```
struct MeshData     { float4 vertex : POSITION; };
struct Interpolators{ float4 vertex : SV_POSITION; };
```

`MeshData` requests exactly one attribute from the vertex buffer: the local-space position, bound by the `POSITION` semantic. Normals, UVs, tangents, and vertex colours are all present in the sphere's vertex buffer, Unity's primitive sphere is a fully-attributed mesh, but the shader does not declare them and so the GPU does not fetch them. This is a deliberate choice: since the fragment shader will not use any of that data, there is no reason to carry it through the pipeline.

`Interpolators` carries only the mandatory `SV_POSITION` output. With no other varyings declared, the rasterizer has nothing to interpolate across the triangle surface.

Vertex shader.

```
Interpolators vert(MeshData v)
{
    Interpolators o;
    o.vertex = UnityObjectToClipPos(v.vertex);
    return o;
}
```

`UnityObjectToClipPos` is a `UnityCG.cginc` wrapper equivalent to:

$$\mathbf{v}_{clip} = M_{proj} \cdot M_{view} \cdot M_{model} \cdot \mathbf{v}_{local} \quad \equiv \quad \texttt{mul(UNITY_MATRIX_MVP, v.vertex)} \tag{199}$$

This single call performs the entire coordinate-space journey from Section 4 in one matrix multiply. The result, written to `SV_POSITION`, is the only value the vertex shader produces. Every vertex is processed independently and in parallel across the GPU's shader cores.

Fragment shader.

```
float4 frag(Interpolators i) : SV_Target
{
    return _Color;
}
```

The fragment shader is a constant function. The parameter `i` is received but never read, there is nothing in `Interpolators` beyond the position, which is consumed by the rasterizer and not forwarded to the fragment shader anyway. Every fragment, regardless of where on the sphere it lies, returns the same `float4` value: the red colour set in the inspector.

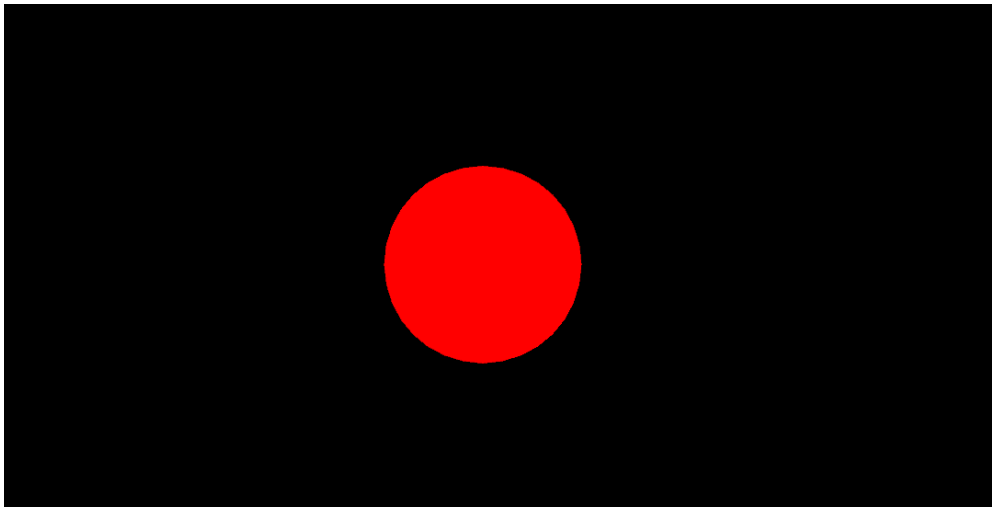


Figure 2: The rendered output. The sphere appears as a flat red disc on a black background. The silhouette is geometrically correct, the rasterizer computed triangle coverage accurately, but without any variation in the fragment output there is no visual information about the curvature of the surface.

Render output. The result (Figure 2) is a perfectly uniform red disc. The silhouette is exactly what the rasterizer computed from the sphere's clip-space triangle coverage: the projection of a sphere onto the image plane is an ellipse, which at this camera position is a circle. The shape is correct. Yet the surface reads as flat because the fragment shader emits no variation in colour. In a lit shader, the per-fragment world-space normal $\hat{\mathbf{n}}$ would modulate the output through the Lambertian term $\max(0, \hat{\mathbf{n}} \cdot \hat{\mathbf{l}})$, darkening fragments where the surface faces away from the light and producing the shading gradient that communicates curvature to the eye. Here, that term is absent entirely.

This is the key property of an unlit shader: the rasterizer runs in full, correctly determining which pixels belong to the surface, but the fragment stage treats every one of those pixels identically. The geometry is present in the pipeline, its clip-space position drove the coverage calculation, it is simply not used in the shading decision.

10.2 Vertex vs Fragment: Choosing Where to Compute

Every arithmetic operation in a shader runs inside one of two loops driven by the GPU. The vertex shader executes once per vertex in the draw call; the fragment shader executes once per fragment, that is, once per pixel covered by the geometry after the rasterizer and depth test. Choosing which stage performs a given computation is therefore a question of how many times each loop body runs.

Counting invocations. Let N_v denote the number of vertices in the draw call and N_f the number of surviving fragments. These two quantities are determined by entirely different factors:

$$N_v = \text{vertex count of the mesh (a property of the asset)} \tag{200}$$

$$N_f \approx W_{screen} \times H_{screen} \times f_{coverage} \tag{201}$$

where $f_{coverage} \in [0, 1]$ is the fraction of the screen the object occupies after projection. N_v is fixed by the mesh regardless of camera distance or screen resolution; N_f scales with the object's projected area.

A concrete estimate. The Unity default sphere (the mesh used in this scene) has $N_v = 515$ vertices. Rendered at 1920×1080 with the sphere covering approximately 20% of the screen:

$$N_f \approx 1920 \times 1080 \times 0.20 \approx 415{,}000 \tag{202}$$

The ratio is:

$$\frac{N_f}{N_v} \approx \frac{415{,}000}{515} \approx 800 \tag{203}$$

The fragment shader runs roughly 800 times more often than the vertex shader for this draw call. Any unit of computation moved from the fragment stage to the vertex stage is therefore approximately 800 times cheaper in terms of total GPU work, without any change to the fragment shader's logic.

What can move to the vertex stage. The rasterizer interpolates varyings across the triangle surface at zero additional GPU cost (the interpolation runs in fixed hardware, not in programmable shader cores). This means any quantity that varies *linearly* across a triangle can be computed per-vertex, stored in a varying, and recovered per-fragment by interpolation with no shader instruction budget spent.

Computations that are safe to move to the vertex stage:

- **Position transforms**: the MVP transform `UnityObjectToClipPos` is already correctly placed in the vertex shader. It is the canonical example of per-vertex work.
- **Normal transformation**: `UnityObjectToWorldNormal` transforms the local-space normal to world space using the inverse-transpose of the model matrix (Section 6.5). The transform itself is linear and correct per-vertex. However, the subsequent `normalize()` call must remain in the fragment shader, because linear interpolation of two unit vectors does not in general produce a unit vector.
- **UV coordinate transforms**: scaling and offsetting UVs (via `TRANSFORM_TEX`) is linear and can be computed per-vertex.
- **Per-vertex (Gouraud) diffuse lighting**: the Lambertian term $\max(0, \hat{\mathbf{n}} \cdot \hat{\mathbf{l}})$ can be evaluated at each vertex and interpolated. This is called *Gouraud shading*. It is cheap, but visually incorrect when the triangle is large on screen, because the Lambertian function is non-linear over the triangle surface and interpolation introduces error.

What must stay in the fragment stage. Some computations produce incorrect results if evaluated per-vertex, regardless of the performance saving:

- **Texture sampling**: `tex2D(_MainTex, uv)` must execute per-fragment. The UV coordinates themselves interpolate correctly, but the GPU texture fetch is a per-fragment instruction. Moving it to the vertex stage is not supported for standard 2D textures.
- **Specular highlights**: the specular term involves $(\hat{\mathbf{r}} \cdot \hat{\mathbf{v}})^s$ where s is the shininess exponent. This is highly non-linear and produces visible faceting artefacts when computed per-vertex, because the highlight's peak can fall entirely inside a triangle and be missed if only the vertex values are sampled.
- **Normal map perturbation**: normal maps modify the surface normal at sub-triangle resolution. The perturbation is read from a texture per-fragment, so the full shading computation must also happen per-fragment.

- **Renormalisation**: as noted above, interpolated normals are not unit vectors. `normalize(IN.normalWS)` in the fragment shader corrects this and costs one reciprocal square root per fragment, but skipping it produces incorrect lighting.

Decision rule. A computation belongs in the vertex shader when two conditions hold simultaneously: the quantity varies linearly (or near-linearly) across the triangle, and the triangle is small enough on screen that the interpolation error is imperceptible. When either condition fails, the computation must move to the fragment shader. In practice, lighting terms that depend on the surface normal are borderline: the normal transform goes per-vertex, the shading evaluation stays per-fragment. The boundary is always the non-linear operation.

Relevance to the current shader. In `Shader1`, the vertex shader performs the MVP transform (correct, one matrix multiply per vertex) and the fragment shader returns `_Color` (a uniform constant, which costs nothing regardless of stage). There is no computation to redistribute. When lighting is introduced, the normal transform will be added to the vertex shader and the $\max(0, \hat{\mathbf{n}} \cdot \hat{\mathbf{l}})$ evaluation will go in the fragment shader, with `normalize` preceding it to correct the interpolated normal length.

10.3 UV Coordinates and Linear Interpolation

In Section 10.1 the fragment shader returned a constant, and the interpolators carried no data beyond the clip-space position required by the rasterizer. This subsection extends the shader with a single varying, the texture coordinate (UV), and uses it to drive a spatial blend between two colours. The resulting render is the first in which the fragment output varies across the surface, and the variation is produced without any texture sampling: the rasterizer's interpolation hardware alone does the work.

What UVs are. UV coordinates are a per-vertex 2D attribute, conventionally written $(u, v) \in [0, 1]^2$, that parameterise the surface of a mesh. Each vertex in the vertex buffer carries a UV value authored by the mesh creator, or generated automatically for Unity primitives. The coordinates have no geometric meaning on their own, they are simply numbers stored alongside the position and normal. Their usefulness comes from what happens downstream. Because they are declared as a varying and interpolated linearly across each triangle by the rasterizer (Section 5.3), every fragment receives a blended UV computed from the three vertex values using barycentric weights. Anything indexed by those coordinates, a texture lookup, an arithmetic formula, a colour blend, then varies smoothly across the surface.

The shader. The updated `Shader1.shader` replaces the single `_Color` property with two colours and two scalar UV transform parameters, declares a UV input attribute, carries it through `Interpolators`, and blends between the two colours in the fragment stage:

```
Shader "Unlit/Shader1"
{
    Properties
    {
        _ColorA ("ColorA", Color) = (0, 0, 0, 0)
        _ColorB ("ColorB", Color) = (1, 1, 1, 1)
        _Scale  ("UV Scale",  float) = 1
        _Offset ("UV Offset", float) = 0
    }

    SubShader
    {
```

```
        Tags { "RenderType"="Opaque" }

        Pass
        {
            CGPROGRAM
            #pragma vertex vert
            #pragma fragment frag

            #include "UnityCG.cginc"

            float4 _ColorA;
            float4 _ColorB;
            float  _Scale;
            float  _Offset;

            struct MeshData
            {
                float4 vertex : POSITION;
                float3 normal : NORMAL;
                float2 uv     : TEXCOORD1;
            };

            struct Interpolators
            {
                float4 vertex : SV_POSITION;
                float3 normal : NORMAL;
                float2 uv     : TEXCOORD0;
            };

            Interpolators vert(MeshData v)
            {
                Interpolators o;
                o.vertex = UnityObjectToClipPos(v.vertex);
                o.normal = UnityObjectToWorldNormal(v.normal);
                o.uv     = (v.uv + _Offset) * _Scale;
                return o;
            }

            float4 frag(Interpolators i) : SV_Target
            {
                float4 outColor = lerp(_ColorA, _ColorB, i.uv.x);
                return outColor;
            }
            ENDCG
        }
    }
}
```

Properties. Four uniforms are declared. `_ColorA` and `_ColorB` are the two endpoints of the colour blend, and `_Scale` and `_Offset` are scalar parameters applied to the UVs before they reach the fragment stage. All four are per-draw-call constants, uploaded once by the CPU into a constant buffer and read identically by every shader invocation, exactly as described for `_Color` in Section 10.1.

Requesting vertex attributes.

```
struct MeshData
```

```
{
    float4 vertex : POSITION;
    float3 normal : NORMAL;
    float2 uv     : TEXCOORD1;
};
```

`MeshData` now declares three attributes instead of the single `POSITION` in Section 10.1. The identifiers after each field name, `POSITION`, `NORMAL`, and `TEXCOORD`n, are HLSL *semantics*: compile-time tags telling the GPU which channel of the vertex buffer to read into that field. The mesh's vertex buffer, described in Section 8.1, stores the attributes in an interleaved layout, and semantics are how the shader names each stream without hard-coding a byte offset. Declaring a field here is what causes the GPU to actually fetch that attribute, anything not listed is ignored at input assembly time and never leaves the vertex buffer.

Of the three attributes requested, only the UV contributes to the fragment output. The normal is fetched, transformed, and carried through `Interpolators` but never read in `frag`. It is present in anticipation of later shading work rather than as a functional requirement of the current effect.

Carrying UVs through Interpolators.

```
struct Interpolators
{
    float4 vertex : SV_POSITION;
    float3 normal : NORMAL;
    float2 uv     : TEXCOORD0;
};
```

The `Interpolators` struct is the contract between the vertex shader and the fragment shader, with the rasterizer sitting in between. Every field except `SV_POSITION` is a *varying*: a value written once per vertex by `vert`, interpolated across the triangle by fixed-function hardware, and delivered to `frag` as the blended per-fragment result. The semantic names on this side of the pipeline (`TEXCOORD0`, `NORMAL`) are internal identifiers used to pair vertex outputs with fragment inputs. They no longer refer to vertex buffer channels, and the specific choice of `TEXCOORD0` for the UV is a convention, not a constraint; any unused `TEXCOORD`n slot would serve equally well.

UV transform in the vertex shader.

```
o.uv = (v.uv + _Offset) * _Scale;
```

The vertex shader applies an affine transform to the incoming UVs before passing them on. Writing s for `_Scale` and o for `_Offset`, the operation is:

$$(u', v') = s \cdot \big((u, v) + o\big) \tag{204}$$

where s and o act uniformly on both components. Scaling stretches or compresses the apparent gradient across the surface, and offsetting slides it along the UV parameterisation. Because this transform is affine in the input UVs, and affine functions commute with barycentric interpolation, evaluating it per-vertex and then interpolating gives exactly the same result as interpolating first and then evaluating per-fragment. This is the archetypal case from Section 10.2 of a computation that belongs in the vertex stage: it varies linearly across the triangle, so moving it there is roughly 800 times cheaper than evaluating it per-fragment at zero visual cost.

The lerp in the fragment shader.

```
float4 outColor = lerp(_ColorA, _ColorB, i.uv.x);
return outColor;
```

HLSL's `lerp(a, b, t)` evaluates the linear blend:

$$\mathrm{lerp}(a, b, t) = (1 - t)\, a + t\, b \tag{205}$$

With `t = i.uv.x`, the per-fragment output colour is:

$$C(u) = (1 - u)\, C_A + u\, C_B \tag{206}$$

The second UV component, `i.uv.y`, is ignored entirely. Every fragment receives its own interpolated value of u, determined by its barycentric position within the triangle and the three vertex-UVs that bound it, and returns the corresponding point on the straight line in RGBA space between `_ColorA` and `_ColorB`. With u clamped to $[0, 1]$ by the default sphere's parameterisation, the output sweeps continuously from C_A at $u = 0$ to C_B at $u = 1$.

Why the surface is no longer flat. In Section 10.1 the fragment shader ignored its input and returned a constant, producing a uniformly coloured disc that gave the eye no information about the shape of the sphere. Here the output still contains no lighting, no normals in the shading equation, and no shadows, yet the surface reads with more structure because the colour now varies with position on the mesh. The variation is a pure function of the UV parameterisation: wherever the sphere's authored UVs map to a small u, the fragment sits near `_ColorA`, and where they map to a large u, it sits near `_ColorB`. The fragment shader remains, as in Section 10.1, a function of its inputs alone with no access to the rest of the scene. The only difference is that those inputs now vary, and the rasterizer has been given a varying to carry.